Dokumentacja: System Overview & Architecture

ForestFire

Autorzy:
Joanna Wojnowska, Antonina Wąsikowska

Kierunek studiów:
Informatyka i Systemy Inteligentne

Przedmiot:
Studio Projektowe - ForestFire

Prowadzący:
prof. dr. hab. Radosław Klimek

AGH
EAIiB
2026

Spis treści

1 System Overview **3**

2 High-Level Architecture **3**
2.1 Przegląd systemu . . . 3
2.2 Główne komponenty . . . 3
2.3 Style komunikacji i protokoły . . . 4
2.4 Konwencje danych . . . 4
2.5 Przepływ danych end-to-end . . . 4
2.6 Uruchamianie jako całość (runtime view) . . . 5

3 System Components **6**
3.1 Forest Environments Visualisator (FFVis) . . . 6
3.2 Fire Processing Backend (FFSup) . . . 6
3.3 Forest Environments Simulator (FFSim) . . . 6
3.4 Forest Environments Configurator (FFConf) . . . 7
3.5 RabbitMQ . . . 7
3.6 MongoDB . . . 8

4 Communication Architecture **10**
4.1 Style komunikacji . . . 10
4.2 Komunikacja synchroniczna – HTTP/REST . . . 10
4.3 Komunikacja asynchroniczna – AMQP / RabbitMQ . . . 10
4.4 Komunikacja strumieniowa – Server-Sent Events . . . 11
4.5 Podsumowanie architektury komunikacyjnej . . . 11

5 Data Flow Overview **11**
5.1 Przepływ danych konfiguracyjnych . . . 12
5.2 Przepływ danych symulacyjnych . . . 12
5.3 Dystrybucja stanu do wizualizacji . . . 13
5.4 Przepływ rozkazów operacyjnych . . . 13
5.5 Charakterystyka przepływu . . . 13

6 Deployment & Runtime Environment **13**
6.1 Środowisko uruchomieniowe . . . 14
6.2 Komponenty runtime . . . 14
6.3 Model wdrożeniowy . . . 14
6.4 Porty i interfejsy sieciowe . . . 15
6.5 Charakterystyka środowiska wykonawczego . . . 15

7 Reliability, Scalability & Observability **15**
7.1 Reliability . . . 16
7.2 Scalability . . . 16
7.3 Observability . . . 16

8 Instalacja i pierwsze uruchomienie **17**
8.1 Wymagania wstępne 17
8.2 Uzyskanie klucza Google Maps API 17
8.3 Konfiguracja pliku środowiskowego (.env) 18
8.4 Uruchomienie systemu 18
8.5 Zatrzymanie i czyszczenie 19

9 Summary **19**

1 System Overview

System **ForestFire** jest rozproszonym systemem informatycznym służącym do monitorowania zagrożeń pożarowych w obszarach leśnych oraz do symulacji rozwoju pożarów i działań ratowniczych. System integruje dane środowiskowe pochodzące z czujników oraz zdarzenia generowane przez moduł symulacyjny, umożliwiając analizę sytuacji pożarowej w czasie rzeczywistym.

ForestFire został zaprojektowany jako system symulacyjno–analityczny wspierający zarówno bieżące monitorowanie stanu lasu, jak i testowanie scenariuszy pożarowych w kontrolowanym środowisku.

System składa się z niezależnych modułów funkcjonalnych, obejmujących backend przetwarzający dane i koordynujący symulację, interfejs wizualizacyjny, moduł konfiguracji scenariuszy, symulator środowiska leśnego oraz bazę danych konfiguracji. Komponenty komunikują się ze sobą w sposób synchroniczny i asynchroniczny, tworząc spójną architekturę rozproszoną.

2 High-Level Architecture

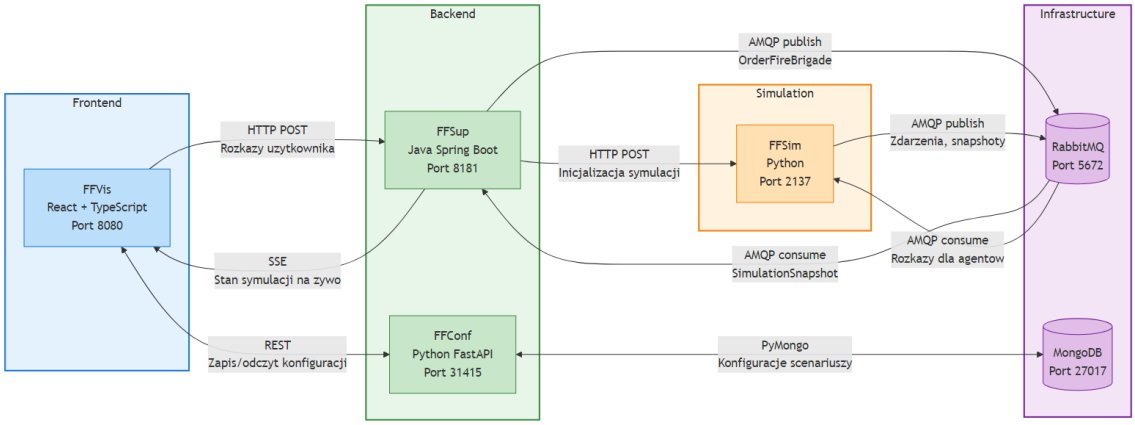
2.1 Przegląd systemu

ForestFire jest systemem rozproszonym, łączącym funkcje monitorowania i symulacji zagrożeń pożarowych. Architektura opiera się na wydzielonych modułach (frontend, backend, symulator, konfigurator) oraz komponentach infrastrukturalnych (broker AMQP i baza danych). Celem takiego podziału jest oddzielenie logiki wizualizacji, przetwarzania i decyzji od warstwy symulacyjnej oraz trwałego przechowywania konfiguracji.

2.2 Główne komponenty

System składa się z następujących elementów:

- **FFVis (Forest Environments Visualisator)** – aplikacja webowa (SPA) uruchamiana w przeglądarce, odpowiedzialna za wizualizację stanu lasu, sektorów, jednostek oraz edycję scenariuszy.
- **FFSup (Fire Processing Backend)** – centralny backend koordynujący przepływ danych, wystawiający REST API oraz strumień SSE z bieżącym stanem symulacji; integruje dane zdarzeniowe z brokera RabbitMQ.
- **FFSim (Forest Environments Simulator)** – moduł symulacyjny generujący pomiary środowiskowe, zdarzenia pożarowe oraz telemetrię agentów; publikuje zdarzenia do RabbitMQ i reaguje na rozkazy operacyjne.
- **FFConf (Forest Environments Configurator)** – serwis CRUD przechowujący konfiguracje scenariuszy (mapy, sektory, czujniki, jednostki) w MongoDB.
- **RabbitMQ** – broker AMQP realizujący asynchroniczną komunikację pomiędzy symulatorem a backendem (zdarzenia i rozkazy).
- **MongoDB** – baza dokumentowa przechowująca konfiguracje scenariuszy (wyłącznie dla FFConf).



Rysunek 1: Ogólny zarys architektury systemu ForestFire (diagram komponentów).

Diagram przedstawia ogólną architekturę systemu składającą się z czterech głównych komponentów: FFSup (backend), FFVis (frontend), FFConf (konfigurator) oraz FFSim (symulator). Komponenty komunikują się poprzez broker RabbitMQ oraz bazę danych MongoDB.

2.3 Style komunikacji i protokoły

W systemie występują trzy podstawowe style komunikacji:

- **HTTP/REST (synchronicznie)** – sterowanie symulacją i operacje usługowe (UI ↔ backend, UI ↔ konfigurator, backend → symulator).
- **SSE (Server-Sent Events, strumieniowo)** – jednokierunkowy strumień bieżącego stanu systemu z backendu do aplikacji webowej (FFSup → FFVis).
- **AMQP (RabbitMQ, asynchronicznie)** – zdarzenia i telemetria z symulatora do backendu (FFSim → RabbitMQ → FFSup) oraz rozkazy operacyjne w drugą stronę (FFSup → RabbitMQ → FFSim).

2.4 Konwencje danych

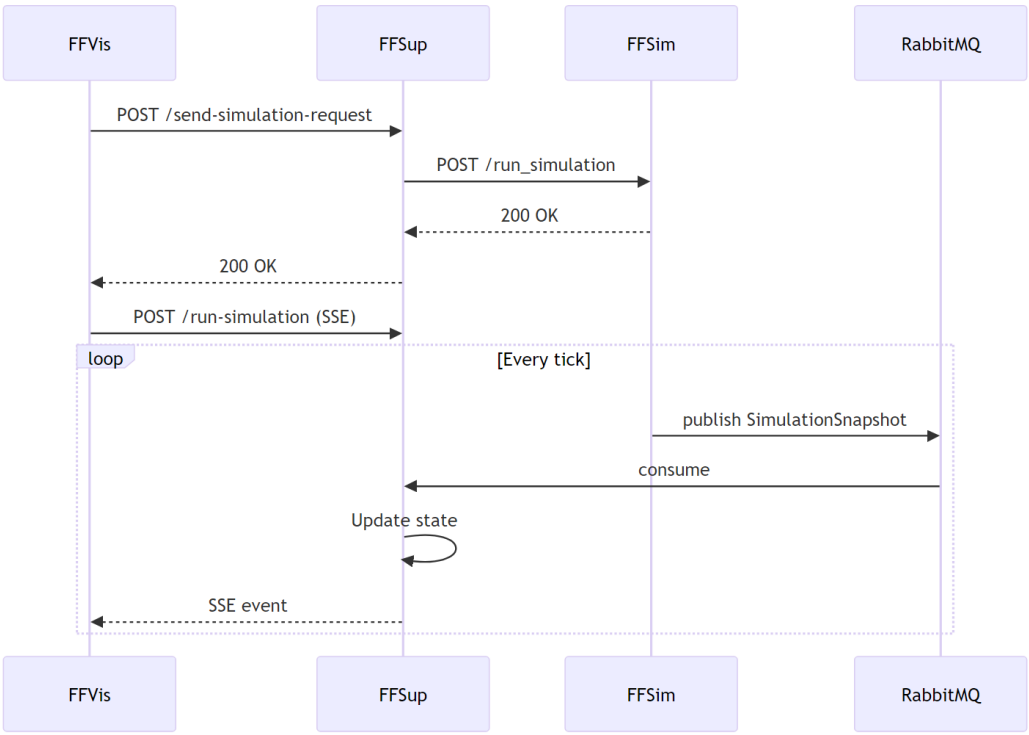
Wszystkie kanały komunikacyjne wykorzystują format **JSON**. Konwencje wspólne dla payloadów:

- kodowanie: UTF-8,
- czas: timestamp jako string,
- lokalizacja: `{"longitude": number, "latitude": number}`,
- wartości typu enum przesyłane jako string (np. `"SEVERE"`, `"PM2_5"`, `"EXTINGUISH"`).

2.5 Przepływ danych end-to-end

Typowy przebieg danych i sterowania wygląda następująco:

1. **Konfiguracja scenariusza:** FFVis pobiera/zapisuje konfiguracje w FFConf przez REST (FFConf utrwala dane w MongoDB).
2. **Start symulacji:** użytkownik uruchamia symulację w UI; FFVis wysyła żądanie do FFSup (REST), a FFSup deleguje uruchomienie do FFSim (REST).
3. **Generowanie zdarzeń:** FFSim w krokach czasowych publikuje zdarzenia środowiskowe, pomiary sensorów i telemetrię agentów do RabbitMQ (AMQP).
4. **Agregacja stanu:** FFSup konsumuje zdarzenia z RabbitMQ, aktualizuje bieżący stan symulacji w pamięci i wylicza pola stanu prezentowane w UI.
5. **Wizualizacja w czasie rzeczywistym:** FFSup emituje zaktualizowany stan do FFVis przez SSE, a frontend aktualizuje mapę i panele danych.
6. **Rozkazy operacyjne:** FFVis wysyła rozkazy (REST) do FFSup, który publikuje je przez RabbitMQ do FFSim (AMQP).



Rysunek 2: Diagram sekwencji — przebieg kroku symulacji

Diagram sekwencji przedstawiający przebieg jednego kroku symulacji. Ilustruje kolejność operacji: aktualizacja ognia, rozprzestrzenianie, ruch brygad i patroli leśniczych.

2.6 Uruchamianie jako całość (runtime view)

System jest uruchamiany w środowisku kontenerowym (Docker/Docker Compose), gdzie każdy moduł działa jako osobna usługa. W typowej konfiguracji lokalnej wykorzystywane są m.in. porty: `8080` (FFVis), `8181` (FFSup), `2137` (FFSim), `31415` (FFConf), `5672` (AMQP RabbitMQ), `15672` (panel RabbitMQ), `27017` (MongoDB).

3 System Components

Niniejszy rozdział przedstawia główne komponenty systemu ForestFire wraz z ich odpowiedzialnościami oraz rolą w architekturze całościowej. Opis ma charakter przeglądowy i koncentruje się na funkcjach realizowanych przez poszczególne moduły, bez wchodzenia w szczegóły implementacyjne.

3.1 Forest Environments Visualisator (FFVis)

FFVis jest aplikacją webową typu Single Page Application (SPA), uruchamianą w przeglądarce użytkownika. Moduł ten pełni rolę interfejsu użytkownika systemu.

Główne odpowiedzialności:

- wizualizacja stanu lasu, sektorów oraz poziomów zagrożenia pożarowego,
- prezentacja położenia i stanów jednostek terenowych (brygad i patroli),
- edycja konfiguracji scenariuszy środowiskowych,
- inicjowanie i sterowanie przebiegiem symulacji,
- odbiór strumienia bieżącego stanu systemu w czasie rzeczywistym.

FFVis komunikuje się z backendem FFSup za pomocą REST oraz Server-Sent Events (SSE), a z modułem konfiguracji FFConf poprzez REST API.

3.2 Fire Processing Backend (FFSup)

FFSup jest centralnym komponentem logicznym systemu. Odpowiada za integrację danych pochodzących z symulatora, koordynację przebiegu symulacji oraz udostępnianie interfejsów dla warstwy wizualizacyjnej.

Główne odpowiedzialności:

- konsumpcja zdarzeń środowiskowych i telemetrii agentów z RabbitMQ,
- utrzymywanie bieżącego stanu symulacji w pamięci operacyjnej,
- emisja strumienia aktualnego stanu systemu do FFVis (SSE),
- obsługa rozkazów operacyjnych wysyłanych z interfejsu użytkownika,
- generowanie rekomendacji decyzyjnych (np. rozmieszczenie brygad).

FFSup pełni rolę pośrednika pomiędzy modułem symulacyjnym a interfejsem użytkownika.

3.3 Forest Environments Simulator (FFSim)

FFSim jest modułem odpowiedzialnym za symulację środowiska leśnego oraz zachowań agentów terenowych. Działa jako niezależna usługa generująca zdarzenia w czasie dyskretnym.

Główne odpowiedzialności:

- symulacja parametrów środowiskowych (temperatura, wilgotność, wiatr, zanieczyszczenia),
- symulacja rozprzestrzeniania się ognia w sektorach,
- obsługa cyklu życia agentów (brygady strażackie, patrole leśne),
- publikacja zdarzeń i pomiarów do brokera RabbitMQ,
- reagowanie na rozkazy operacyjne wysyłane przez backend.

FFSim nie przechowuje stanu trwałego — wszystkie generowane dane mają charakter zdarzeniowy.

3.4 Forest Environments Configurator (FFConf)

FFConf jest serwisem konfiguracyjnym odpowiedzialnym za trwałe przechowywanie oraz edycję scenariuszy środowiskowych.

Główne odpowiedzialności:

- zarządzanie konfiguracjami map, sektorów i granic obszarów,
- obsługa definicji czujników, kamer oraz jednostek terenowych,
- udostępnianie interfejsu REST do operacji CRUD,
- walidacja spójności konfiguracji przed zapisem.

FFConf jest jedynym komponentem posiadającym bezpośredni dostęp do bazy danych MongoDB.

3.5 RabbitMQ

RabbitMQ pełni rolę brokera wiadomości w architekturze systemu. Umożliwia asynchroniczną komunikację pomiędzy modułem symulacyjnym a backendem.

Główne odpowiedzialności:

- dystrybucja zdarzeń środowiskowych i telemetrii agentów,
- pośrednictwo w przekazywaniu rozkazów operacyjnych,
- separacja czasowa i logiczna producentów oraz konsumentów danych.

Zastosowanie brokera zwiększa odporność systemu na przeciążenia oraz awarie pojedynczych komponentów.

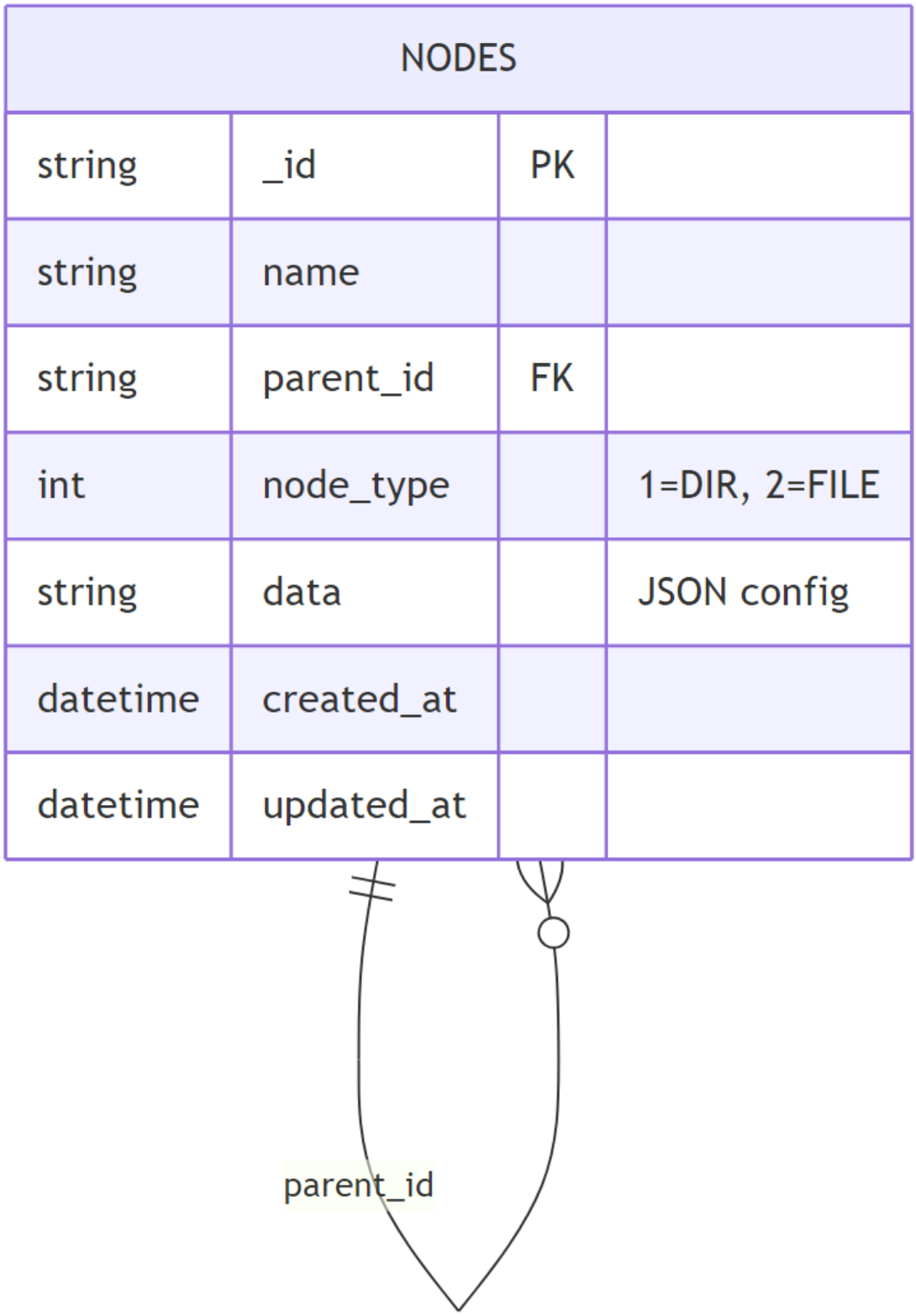
3.6 MongoDB

MongoDB jest nierelacyjną bazą danych wykorzystywaną do trwałego przechowywania konfiguracji systemu.

Zakres odpowiedzialności:

- przechowywanie dokumentów konfiguracji scenariuszy,
- składowanie definicji map, sektorów i obiektów środowiskowych,
- zapewnienie elastycznego modelu danych dla konfiguracji.

Baza danych nie przechowuje danych symulacyjnych ani wyników przebiegu symulacji.



Rysunek 3: Schemat kolekcji bazy danych MongoDB

Struktura kolekcji w bazie MongoDB. MongoDB przechowuje wyłącznie konfiguracje scenariuszy (nie dane symulacyjne).

4 Communication Architecture

Architektura komunikacyjna systemu ForestFire została zaprojektowana w sposób umożliwiający luźne powiązanie komponentów, skalowalność oraz odporność na błędy. System wykorzystuje kombinację komunikacji synchronicznej, asynchronicznej oraz strumieniowej, dostosowanej do charakteru wymienianych danych.

4.1 Style komunikacji

W systemie zastosowano trzy podstawowe style komunikacji:

- **Komunikacja synchroniczna (HTTP/REST)** – wykorzystywana do operacji sterujących, zarządzania konfiguracją oraz inicjowania działań.
- **Komunikacja asynchroniczna (AMQP / RabbitMQ)** – wykorzystywana do przesyłania zdarzeń środowiskowych, telemetrii agentów oraz rozkazów operacyjnych.
- **Komunikacja strumieniowa (Server-Sent Events, SSE)** – wykorzystywana do przekazywania bieżącego stanu systemu w czasie rzeczywistym do interfejsu wizualizacyjnego.

4.2 Komunikacja synchroniczna – HTTP/REST

Komunikacja synchroniczna oparta o protokół HTTP i styl REST wykorzystywana jest w następujących przypadkach:

- interakcja interfejsu użytkownika z backendem (sterowanie symulacją, wydawanie rozkazów),
- zarządzanie konfiguracją scenariuszy (FFVis $\leftrightarrow$ FFConf),
- inicjowanie i zatrzymywanie symulacji (FFSup $\rightarrow$ FFSim).

Ten model komunikacji zapewnia prostotę integracji, jednoznaczność odpowiedzi oraz łatwość debugowania w warstwie aplikacyjnej.

4.3 Komunikacja asynchroniczna – AMQP / RabbitMQ

Asynchroniczna komunikacja pomiędzy modułami realizowana jest z wykorzystaniem brokera RabbitMQ oraz protokołu AMQP.

Broker pełni rolę pośrednika pomiędzy producentami i konsumentami zdarzeń, umożliwiając:

- separację czasową nadawców i odbiorców,
- buforowanie komunikatów,
- niezależne skalowanie komponentów.

Ten kanał komunikacji wykorzystywany jest do:

- przesyłania danych środowiskowych generowanych przez symulator (FFSim $\rightarrow$ FFSup),

- przekazywania telemetrii i stanów agentów,
- wysyłania rozkazów operacyjnych z backendu do symulatora (FFSup → FFSim).

Zastosowanie komunikacji asynchronicznej zwiększa odporność systemu na przeciążenia oraz awarie pojedynczych komponentów.

4.4 Komunikacja strumieniowa – Server-Sent Events

Do przekazywania bieżącego stanu symulacji do interfejsu użytkownika zastosowano mechanizm Server-Sent Events (SSE).

W tym modelu:

- backend (FFSup) emituje jednokierunkowy strumień zdarzeń,
- interfejs wizualizacyjny (FFVis) odbiera i przetwarza aktualizacje w czasie rzeczywistym.

Rozwiązanie to pozwala na:

- ciągłą aktualizację mapy i paneli danych,
- eliminację konieczności cyklicznego odpytywania backendu,
- redukcję narzutu komunikacyjnego po stronie klienta.

4.5 Podsumowanie architektury komunikacyjnej

Połączenie trzech stylów komunikacji pozwala na dopasowanie mechanizmu wymiany danych do charakteru operacji:

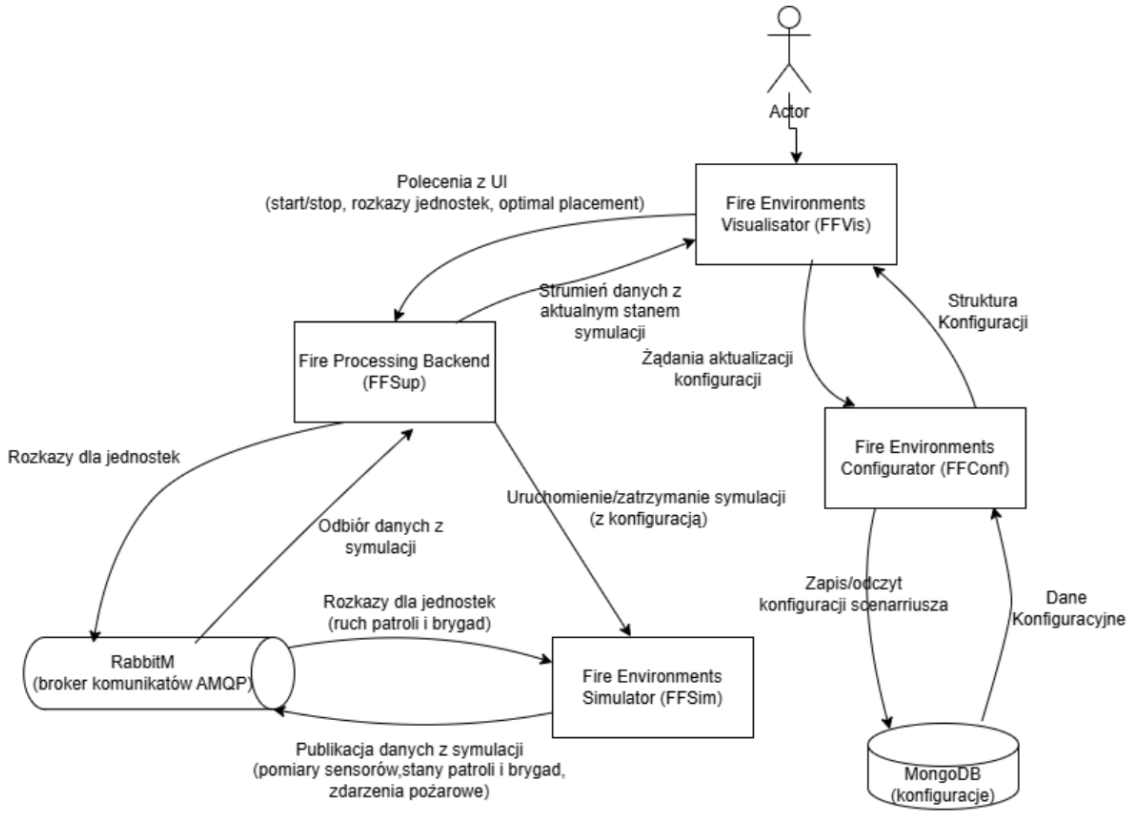
- REST – operacje sterujące i konfiguracyjne,
- AMQP – zdarzenia i komunikacja domenowa,
- SSE – wizualizacja stanu systemu w czasie rzeczywistym.

Takie podejście zapewnia spójność architektury, skalowalność oraz czytelny podział odpowiedzialności pomiędzy komponentami systemu ForestFire.

5 Data Flow Overview

System **ForestFire** realizuje przetwarzanie danych w architekturze zdarzeniowej, łączącej komunikację synchroniczną (REST), asynchroniczną (AMQP) oraz strumieniową (SSE). Przepływ danych obejmuje zarówno konfigurację środowiska, jak i bieżącą wymianę informacji o stanie symulowanego lasu, jednostkach terenowych oraz zagrożeniach pożarowych.

Na rysunku 4 przedstawiono wysokopoziomowy diagram przepływu danych pomiędzy głównymi komponentami systemu, ilustrujący podstawowe kanały komunikacyjne i kierunki wymiany informacji.



Rysunek 4: Diagram przepływu danych w systemie

5.1 Przepływ danych konfiguracyjnych

Proces rozpoczyna się w module wizualizacyjnym **FFVis**, gdzie użytkownik tworzy lub edytuje konfigurację środowiska leśnego (mapy, sektory, czujniki, kamery, jednostki terenowe). Dane te są zapisywane w sposób trwały w bazie **MongoDB** za pośrednictwem modułu **FFConf**.

Podczas uruchamiania symulacji skonfigurowany scenariusz jest przesyłany z **FFVis** do backendu **FFSup**, który pełni rolę punktu integracyjnego i przekazuje konfigurację do modułu symulacyjnego **FFSim**. Na tej podstawie inicjalizowane jest środowisko symulacji.

5.2 Przepływ danych symulacyjnych

W trakcie działania symulacji moduł **FFSim** generuje zdarzenia reprezentujące:

- odczyty czujników środowiskowych,
- stany i pozycje patroli leśnych oraz brygad strażackich,
- zmiany stanu pożaru w sektorach,
- sygnały z kamer wykrywających dym.

Zdarzenia te są publikowane asynchronicznie do brokera **RabbitMQ**. Backend **FFSup** konsumuje komunikaty, aktualizuje globalny stan symulacji oraz wykonuje logikę decyzyjną, w tym obsługę alarmów pożarowych i analizę zagrożeń.

5.3 Dystrybucja stanu do wizualizacji

Aktualny stan systemu jest cyklicznie emitowany przez **FFSup** do modułu wizualizacyjnego **FFVis** za pomocą mechanizmu **Server-Sent Events (SSE)**. Strumień danych zawiera zagregowaną migawkę sytuacji:

- stan sektorów lasu,
- poziom zagrożenia i pożaru,
- lokalizacje i stany jednostek terenowych.

Frontend wykorzystuje ten strumień do bieżącej aktualizacji mapy i paneli informacyjnych bez konieczności wykonywania dodatkowych zapytań.

5.4 Przepływ rozkazów operacyjnych

Decyzje użytkownika (np. wysłanie brygady do gaszenia pożaru lub patrolu do sektora) są przekazywane z **FFVis** do **FFSup** poprzez interfejs REST. Backend publikuje odpowiednie rozkazy do **RabbitMQ**, skąd są one konsumowane przez **FFSim** i realizowane przez agentów w środowisku symulacyjnym.

5.5 Charakterystyka przepływu

Przepływ danych w systemie ma charakter:

- **asynchroniczny** – zdarzenia symulacyjne i telemetria agentów,
- **reaktywny** – backend reaguje na zdarzenia bez blokowania,
- **strumieniowy** – ciągła dystrybucja stanu do interfejsu użytkownika,
- **luźno powiązany** – komponenty mogą być rozwijane i skalowane niezależnie.

6 Deployment & Runtime Environment

System **ForestFire** został zaprojektowany jako zestaw niezależnych usług uruchamianych w środowisku kontenerowym. Architektura runtime opiera się na modelu mikroserwisowym, w którym każdy komponent systemu działa jako osobny proces, komunikujący się z pozostałymi poprzez zdefiniowane interfejsy sieciowe.

6.1 Środowisko uruchomieniowe

Wszystkie moduły systemu są uruchamiane w kontenerach Docker, co zapewnia:

- spójność środowiska wykonawczego,
- łatwość wdrażania i aktualizacji,
- izolację procesów,
- możliwość niezależnego restartu komponentów.

System może być uruchamiany na systemach operacyjnych Linux, macOS oraz Windows (z wykorzystaniem Docker Desktop). Komponenty komunikują się w ramach wspólnej sieci Dockera, wykorzystując nazwy usług jako adresy hostów.

6.2 Komponenty runtime

W środowisku uruchomieniowym działają następujące usługi:

- **FFVis (Forest Environments Visualisator)** – aplikacja frontendowa typu SPA, uruchamiana w przeglądarce użytkownika, odpowiedzialna za wizualizację stanu systemu i konfigurację scenariuszy,
- **FFSup (Fire Processing Backend)** – centralny backend systemu, integrujący dane z symulatora, realizujący logikę decyzyjną oraz udostępniający API REST i SSE,
- **FFSim (Forest Environments Simulator)** – moduł symulacyjny odpowiedzialny za generowanie zdarzeń środowiskowych, ruch agentów i rozprzestrzenianie się pożaru,
- **FFConf (Forest Environments Configurator)** – serwis konfiguracyjny realizujący operacje CRUD na scenariuszach i przechowujący dane w bazie MongoDB,
- **RabbitMQ** – broker wiadomości AMQP obsługujący asynchroniczną komunikację zdarzeniową,
- **MongoDB** – nierelacyjna baza danych przechowująca trwałe konfiguracje środowiska.

Każdy z komponentów może być skalowany i aktualizowany niezależnie od pozostałych.

6.3 Model wdrożeniowy

System jest wdrażany jako całość za pomocą pliku `docker-compose.yml`, który definiuje:

- obrazy kontenerów poszczególnych usług,
- mapowanie portów,
- zależności pomiędzy usługami,
- wolumeny dla danych trwałych,

- wspólną sieć logiczną.

Baza danych MongoDB korzysta z wolumenu Dockerowego, co zapewnia trwałość konfiguracji scenariuszy pomiędzy restartami systemu. Pozostałe komponenty (FFSup, FFSim, FFVis) są bezstanowe — ich restart nie wpływa na zapisane dane konfiguracyjne.

6.4 Porty i interfejsy sieciowe

W typowym środowisku lokalnym system udostępnia następujące interfejsy sieciowe:

- Frontend (FFVis): `http://localhost:8080`
- Backend REST/SSE (FFSup): `http://localhost:8181`
- Symulator (FFSim): `http://localhost:2137`
- Konfigurator (FFConf): `http://localhost:31415`
- RabbitMQ (AMQP): port `5672`
- RabbitMQ Management UI: `http://localhost:15672`
- MongoDB: port `27017`

6.5 Charakterystyka środowiska wykonawczego

Środowisko runtime systemu ForestFire charakteryzuje się:

- **luźnym powiązaniem komponentów** – komunikacja przez REST, AMQP i SSE,
- **asynchronicznym przetwarzaniem zdarzeń** – wykorzystanie RabbitMQ,
- **reaktywnym backendem** – przetwarzanie strumieni zdarzeń bez blokowania,
- **brakiem stanu trwałego symulacji** – stan świata utrzymywany jest wyłącznie w pamięci backendu,
- **odpornością na restart usług** – możliwość restartu pojedynczych komponentów bez zatrzymywania całego systemu.

Architektura ta umożliwia dalszą rozbudowę systemu, w tym wdrażanie dodatkowych modułów analitycznych lub agentów decyzyjnych, bez konieczności przebudowy całego środowiska wykonawczego.

7 Reliability, Scalability & Observability

Architektura systemu **ForestFire** została zaprojektowana z uwzględnieniem odporności na błędy, możliwości skalowania oraz obserwowalności operacyjnej. Rozwiązania zastosowane w systemie wynikają bezpośrednio z mikroserwisowej struktury systemu oraz zdarzeniowego modelu komunikacji.

7.1 Reliability

Niezawodność systemu osiągana jest poprzez separację odpowiedzialności pomiędzy komponentami oraz asynchroniczny model wymiany danych.

Każdy z głównych modułów systemu (FFVis, FFSup, FFSim, FFConf) działa jako niezależna usługa, co ogranicza wpływ awarii pojedynczego komponentu na całość systemu. W przypadku restartu jednego z modułów pozostałe usługi mogą kontynuować działanie.

Asynchroniczna komunikacja pomiędzy backendem a symulatorem, realizowana przez brokera **RabbitMQ**, umożliwia buforowanie zdarzeń i redukuje ryzyko utraty danych w przypadku chwilowej niedostępności odbiorcy. Backend **FFSup** przetwarza zdarzenia w sposób reaktywny, a błędy deserializacji lub przetwarzania są rejestrowane bez przerywania działania strumieni.

Stan symulacji utrzymywany jest w pamięci operacyjnej backendu i emitowany cyklicznie do interfejsu użytkownika. Takie podejście upraszcza model spójności i eliminuje problemy związane z trwałym zapisem szybkozmiennego stanu świata symulacji.

7.2 Scalability

System ForestFire został zaprojektowany w sposób umożliwiający skalowanie horyzontalne wybranych komponentów.

Dzięki zastosowaniu architektury mikroserwisowej oraz komunikacji zdarzeniowej, moduły backendowe i symulacyjne mogą być replikowane jako niezależne instancje kontenerów. Broker **RabbitMQ** pełni rolę centralnego punktu dystrybucji zdarzeń, umożliwiając obsługę rosnącej liczby komunikatów bez ścisłego powiązania pomiędzy producentami i konsumentami danych.

Warstwa wizualizacji (FFVis) jest skalowalna w sposób naturalny — każda instancja interfejsu użytkownika konsumuje strumień danych niezależnie. Serwis konfiguracji (FFConf) może być skalowany wraz z bazą MongoDB, która w przyszłości umożliwia replikację danych i poziome rozszerzanie pojemności.

7.3 Observability

Obserwowalność systemu ForestFire oparta jest na centralnym logowaniu, monitorowaniu komponentów oraz analizie zdarzeń przepływających przez system.

Wszystkie moduły systemu generują logi aplikacyjne kierowane do standardowego wyjścia kontenerów Dockera. Logi te obejmują m.in.:

- przetwarzanie zdarzeń i decyzji operacyjnych w module **FFSup**,
- przebieg symulacji, cykl życia agentów oraz generację zdarzeń w module **FFSim**,
- operacje konfiguracyjne w module **FFConf**,
- komunikaty diagnostyczne warstwy wizualizacji **FFVis**.

Logi kontenerów są zbierane przez system **Loki**, który pełni rolę centralnego agregatora logów w architekturze. Dane logowe są następnie wizualizowane i analizowane w narzędziu **Grafana**, umożliwiając:

- filtrowanie logów według usług, poziomów i znaczników czasowych,
- korelację zdarzeń pomiędzy komponentami systemu,

- szybką identyfikację błędów oraz wąskich gardeł,
- analizę zachowania systemu w czasie rzeczywistym.

Dodatkowym elementem obserwowalności jest interfejs administracyjny brokera **RabbitMQ**, który pozwala monitorować stan kolejek, liczbę wiadomości, połączeń oraz kondycję kanałów komunikacyjnych.

Zastosowana architektura umożliwia dalsze rozszerzenie warstwy obserwowalności o metryki wydajnościowe, alerting oraz distributed tracing, bez konieczności ingerencji w logikę biznesową systemu.

8 Instalacja i pierwsze uruchomienie

W tej sekcji opisano procedurę instalacji systemu ForestFire od podstaw, włącznie z wymaganiami wstępnymi, konfiguracją kluczy API oraz uruchomieniem wszystkich komponentów.

8.1 Wymagania wstępne

Przed rozpoczęciem instalacji należy upewnić się, że na komputerze zainstalowane są następujące narzędzia:

- **Docker Desktop** – środowisko kontenerowe wymagane do uruchomienia wszystkich usług systemu. Dostępne pod adresem: `https://www.docker.com/products/docker-desktop/`

Po zainstalowaniu Docker Desktop należy upewnić się, że usługa Docker jest uruchomiona (ikona w zasobniku systemowym powinna być aktywna).

8.2 Uzyskanie klucza Google Maps API

Moduł wizualizacji (FFVis) wymaga klucza API Google Maps do wyświetlania interaktywnej mapy. Aby uzyskać klucz:

1. Wejdź na stronę **Google Cloud Console**: `https://console.cloud.google.com/`
2. Zaloguj się kontem Google (lub utwórz nowe konto).
3. Utwórz nowy projekt:
   - Kliknij na selektor projektów w górnym pasku (obok napisu "Google Cloud").
   - Wybierz **"Nowy projekt"** i nadaj mu nazwę (np. "ForestFire").
   - Kliknij **"Utwórz"**.
4. Włącz API Maps JavaScript:
   - W menu bocznym wybierz **"APIs & Services"** → **"Library"**.
   - Wyszukaj **"Maps JavaScript API"** i kliknij na wynik.
   - Kliknij przycisk **"Enable"** (Włącz).

5. Utwórz klucz API:
   - W menu bocznym wybierz **"APIs & Services"** → **"Credentials"**.
   - Kliknij **"+ Create Credentials"** → **"API Key"**.
   - Skopiuj wygenerowany klucz (ciąg znaków zaczynający się od `AIza...`).
6. (Opcjonalnie) Utwórz Map ID:
   - W menu wybierz **"Google Maps Platform"** → **"Map Management"**.
   - Kliknij **"Create Map ID"**, wybierz typ "JavaScript"i zapisz identyfikator.

Uwaga: Google wymaga podpięcia metody płatności do konta, jednak oferuje darmowy limit $200/miesiąc, który jest wystarczający dla celów deweloperskich i testowych.

8.3 Konfiguracja pliku środowiskowego (.env)

System wymaga pliku konfiguracyjnego `.env` w katalogu `fire-visualization/`. Plik ten zawiera klucze API i nie powinien być udostępniany publicznie.

1. Przejdź do katalogu `fire-visualization/`.
2. Utwórz plik `.env` na podstawie szablonu `.env.example`:

```
VITE_GOOGLE_API_KEY=AIzaSy...twoj_klucz_api...
VITE_GOOGLE_MAP_ID_MAIN_MAP=twoj_map_id
NODE_ENV=development
```

Gdzie:

- `VITE_GOOGLE_API_KEY` – klucz API uzyskany z Google Cloud Console (wymagany).
- `VITE_GOOGLE_MAP_ID_MAIN_MAP` – identyfikator mapy z Google Maps Platform (opcjonalny, można pominąć).

8.4 Uruchomienie systemu

Po skonfigurowaniu pliku `.env` można uruchomić cały system za pomocą Docker Compose:

```
# 1. Upewnij sie, ze plik .env istnieje
# (fire-visualization/.env z kluczem Google Maps API)

# 2. Uruchom wszystkie kontenery
docker-compose up -d

# 3. Sprawdz status uslug
docker-compose ps

# 4. Otworz przegladarke
# Frontend: http://localhost:8080
# Backend:  http://localhost:8181
# RabbitMQ: http://localhost:15672 (guest/guest)
```

Pierwsze uruchomienie może potrwać kilka minut ze względu na pobieranie obrazów Docker i budowanie kontenerów.

8.5 Zatrzymanie i czyszczenie

```
# Zatrzymanie systemu (zachowuje dane)
docker-compose down

# Zatrzymanie i usuniecie danych (pelny reset)
docker-compose down -v
```

9 Summary

Dokument przedstawia architekturę systemu **ForestFire** z perspektywy technicznej, koncentrując się na jego strukturze, sposobie komunikacji, przepływie danych oraz środowisku uruchomieniowym. System został zaprojektowany jako rozproszone, mikroserwisowe rozwiązanie, łączące symulację środowiska leśnego z przetwarzaniem zdarzeniowym i wizualizacją danych w czasie rzeczywistym.

Architektura ForestFire opiera się na wyraźnym podziale odpowiedzialności pomiędzy moduły wizualizacji, backend integracyjny, silnik symulacyjny oraz serwis konfiguracji. Komunikacja pomiędzy komponentami realizowana jest z wykorzystaniem protokołów REST, AMQP oraz mechanizmu Server-Sent Events, co pozwala na efektywne i skalowalne przetwarzanie danych.

Zastosowanie konteneryzacji oraz narzędzi takich jak RabbitMQ, MongoDB, Loki i Grafana umożliwia niezależne wdrażanie usług, centralne monitorowanie oraz analizę zachowania systemu w środowisku runtime. Przyjęte rozwiązania zapewniają odporność na błędy, możliwość dalszego skalowania oraz dobrą obserwowalność operacyjną.

Przedstawiona architektura stanowi solidną podstawę do dalszego rozwoju systemu, w tym rozbudowy logiki decyzyjnej, integracji zaawansowanych algorytmów analitycznych oraz rozszerzenia warstwy monitoringu i automatyzacji.